

The Topological Trouble With Transformers

Michael C. Mozer · Shoaib Ahmed Siddiqui · Rosanne Liu

ABSTRACT

Transformers encode structure in sequences via an expanding contextual history. However, their purely feedforward architecture fundamentally limits dynamic state tracking. State tracking—the iterative updating of latent variables reflecting an evolving environment—involves inherently sequential dependencies that feedforward networks struggle to maintain. Consequently, feedforward models push evolving state representations deeper into their layer stack with each new input step, rendering information inaccessible in shallow layers and ultimately exhausting the model’s depth. While this depth limit can be bypassed by dynamic depth models and by explicit or latent thinking that externalizes state representations, these solutions are computationally and memory inefficient. In this article, we argue that temporally extended cognition requires refocusing from explicit thought traces to implicit activation dynamics via recurrent architectures. We introduce a taxonomy of recurrent and continuous-thought transformer architectures, categorizing them by their recurrence axis (depth versus step) and their ratio of input tokens to recurrence steps. Finally, we outline promising research directions, including enhanced state-space models and coarse-grained recurrence, to better integrate state tracking into modern foundation models.

1 1 [1] 1

1 Introduction

Progress in understanding human cognition has resulted from conceptualizing the brain as a dynamical system. In terms of its hardware, the physical brain is composed of billions of interacting neurons whose collective behavior is inherently dynamical. In terms of its function, the emergent mind can be usefully modeled as a dynamical process with a high-dimensional state, s , that evolves over time, modulated by external stimuli, x . These levels can be bridged by formalizing the state progression as $s_t = f(s_{t-1}, x_t)$, assuming discrete time t .

From this perspective, an ideal architecture for modeling temporally extended cognition would be a recurrent neural network (RNN), which explicitly performs such a state-update operation. In principle, gradient-based training procedures might discover the function f from data such that the important input signals would be integrated into the state representation and held until later required. The appeal of RNNs was somewhat dampened in the 1990s by the inherent

limitations of gradient-based training [30, 32, 65].

Until the transformer [88] came along, feedforward nets did not seem like a viable approach to replicating human thought and reasoning. The transformer, with an audaciously long context window, retains all information in its history, often postponing the selection of relevant data until required for inference [58]. In contrast, RNNs filter information as it arrives into a bottlenecked state representation [31]. This article is about what can go wrong with the transformer’s strategy and approaches that can address its limitations.

2 State tracking

Tracking the evolving world state is an essential ingredient for language understanding and reasoning, regardless of how tracking is achieved. The transformer’s strategy often leverages its capacity to retrieve static, previously observed information from a context window. The attention mechanism of transformers is highly effective at retrieving past tokens [68]. However, this lookup mechanism is conceptually distinct from the explicit maintenance of a dynamic state—the iterative, inherently sequential updating of latent variables that reflect a changing environment. The term *belief state* is often used to refer to this compact, sufficient summary of the knowledge an AI agent has about its environment [16, 41]. Belief state can be a set of facts or it can be a probability distribution over possible worlds.

To illustrate, consider the game of twenty questions. Each answer to a question narrows the hypothesis space, and each subsequent question should be designed to shrink the hypothesis space further. In a game where one is asked to guess a number, if one is told that the number is larger than 50, it would make no sense to follow up it with a guess of 25. And on the flip side, if one is asked to think of a number between 1 and 100 and respond ‘higher’ or ‘lower’ to guesses, maintaining the state—the hidden number—is critical to preventing inconsistent answers. Yet, here is a trace from Gemini 3 (Fast) revealing a failure mode of models:¹

```
User: Think of a number between 1 and 100 and I'll guess. Please respond  
only with "higher" or "lower" or "you got it". My first guess is 60.  
Model: lower  
User: 41  
Model: lower  
User: 70  
Model: higher
```

Even though the model cannot generate a random number internally, it can play the game without actually having a number in mind simply by choosing responses consistent with its previous responses. However, consistency requires tracking the valid range. Gemini 3 Thinking

¹This example was originally suggested by Gamal ElSayed circa 2020, still causing some modern foundation models to fail. Other models may have been trained to address this particular failure mode. Baldelli et al. [6] formally address the failure of standard models to reliably maintain a consistent hidden state over sequential interactions.

does generate an explicit target, but then it fails to make use of the generated target even though it is also a part of the input token stream in this case:

User: Think of a number between 1 and 100 and I'll guess. Please respond only with "higher" or "lower" or "you got it". My first guess is 60.
 Model (thinking): *I've begun designing the core game mechanics. I've successfully established the need for a number within the range of 1 to 100. The user's initial input, which is 60, is noted. Now I've internally selected the hidden number, choosing 42. Since 60 is higher than 42, my next step is to correctly communicate the response "lower".*
 Model: lower
 User: 42
 Model: lower

Beyond games, state tracking is essential to understanding the ever-changing world, the structure of arguments, and social interactions. State tracking failures in foundation models lead to loss of coherence in multi-turn conversations [46], inefficiency in information gathering [79], and breakdowns in communication and cooperation in multi-agent settings [18, 44].

Without proper state tracking, models flip-flop in their interpretations and fail to detect their inconsistencies, e.g., the meaning of a polysemous word [47]:²

User: Fred took the day off work and pulled out his fishing pole. He drove to the bank. When he reaches the bank, should he wear boots or sandals?
 Model: While sandals may be more comfortable for a casual day, they are not the best choice for fishing, especially at a river bank. Here's why:
 ...
 User: Is it likely that Fred will find an ATM at this bank?
 Model: It is highly likely that Fred will find an ATM at the bank. Most banks, especially those located near bodies of water popular for fishing, have ATMs on-site for customer convenience.

In this example, the model jumps from one interpretation of bank to the other without a human-like acknowledgment of the reversal (e.g., “Oh wait, I misinterpreted. You must have been talking about a financial institution.”). We consider this flip-flop a failure to track the world situation, the model’s earlier responses, and the listener’s expectations.

While tracking Fred’s location in the above example should not be challenging, in the most general case, it is untenable for models—and people—to maintain and track probabilistic belief states over all environmental possibilities because the distributions explode in dimensionality. People adopt heuristics such as sampling [e.g., 90], collapsing distributions into prototypical

²The example that follows was produced by Gemini 2.5 Flash in 2025. Although the model sometimes responded correctly, and newer and more powerful models are much less susceptible to this error, the example reveals a fundamental limitation of the core architecture.

cases [87], or forming concrete mental models most consistent with premises [39], like a MAP estimate. Nonetheless, even finite-memory, deterministic state tracking can be unreliable in a transformer decoder.

We will use a schematic to explain the challenge of state tracking. In Figure 1a, we depict the transformer with input steps shown along the horizontal axis and blocks (or layers) of the transformer along the vertical axis. Activation propagates from bottom to top (shallow to deep layers). For three selected blocks, we use color to indicate the functional connectivity of a causal transformer: activation in a block is influenced by all blocks immediately below and below to the left. In Figure 1b, we depict the flow of state information, indicated by the green rectangles. The integration of the state representation and a new input (purple arrows) leads to a new state representation, $s_t = f(s_{t-1}, x_t)$. Because the architecture is feedforward, s_t must lie deeper in the stack than s_{t-1} , eventually topping out of the model. Figure 2 shows examples from several recent articles matching this upward activation flow. This flow can make the integration of information over the sequence unreliable [6, 9, 26, 47, 79, 89].

Not every state-tracking problem requires depth linear in the number of layers. Merrill and Sabharwal [61] prove the necessity and sufficiency of $\log n$ layers to recognize regular language strings of up to length n and graph-connectivity problems with n vertices. However, this proof addresses only the constructability of solutions, not their learnability.

In practice, many researchers have identified clever solutions obtained by training depth-limited models on specific finite sequence-length problems [49, 72, 73, 81]. Essentially, when the state update function, f is of a certain form, the sequence of state updates may be composed into a simpler one-step function, e.g., there exists a function g computable by a transformer layer such that

$$s_t = f(\dots f(f(s_0, x_1), x_2), \dots, x_t) = g(s_0, x_1, \dots, x_t).$$

Training losses have been proposed that aim to steer models toward such solutions, to the

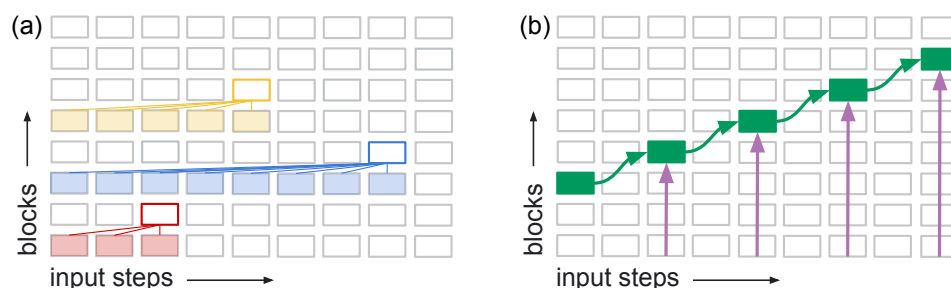


FIGURE 1 (a) Schematic depiction of transformer decoder architecture with input steps along horizontal axis and blocks (layers) along vertical axis. Activation flows upwards, from shallow to deep layers. For three selected blocks outlined in color, the functional input connectivity of a causal model is shown by colored lines and shading of the input sources. (b) The propagation of state representations, indicated by the green rectangles. In cases where these representations depend on previous states and current inputs, the state tracking capability of a feedforward transformer is fundamentally limited by the depth of the model.

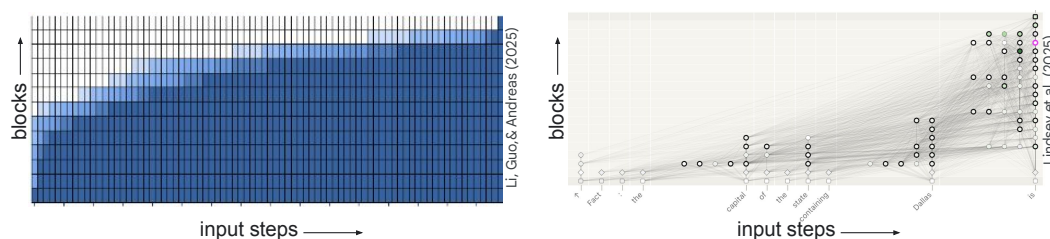


FIGURE 2 Two examples from the literature: Li et al. [49] and Lindsey et al. [54]. The details of each Figure are not critical, but in each case, the upward flow of information to deeper layers is depicted.

extent they exist exactly or approximately [34, 35, 85]. However, when belief-state cascades push deeper and deeper into a network, computational limitations arise because the resulting representations are unavailable to shallower layers. To illustrate, Figure 3 depicts the processing of the bank dialog. Using a technique called Patchscopes [23], Lepori et al. [47] observed that the embedding of the polysemous word ‘bank’ was ambiguous (i.e., a mixture of money bank vs. river bank) at shallow layers of the network, but deep in the network, the model selected the river bank interpretation. The previous sentences provided context (e.g., ‘took the day off work’, ‘fishing pole’) to support one interpretation over the other. In the Figure, we depict this contextualization as occurring at the sixth block of the transformer stack, which means that when subsequent tokens are processed, the disambiguation is not available in blocks 1-5. Lepori et al. [47] show that this delayed disambiguation leads to downstream errors whenever the response generation outpaces the model’s internal semantic convergence, such as when the model forms its yes/no response to the question about an ATM at the bank.

If reasonable models make errors when the inference cascade is just two steps (Lepori et al. studied Gemma2-9B), it should be no surprise that models produce more severe failures in comprehending extended multi-agent conversations.

One solution to the depth dilemma is chain-of-thought style “thinking” where the model recasts a deep representation as one or more output tokens, which are then available to the model on its input [91]. For ordinary, step-by-step microcognition, this solution is a cop out. Inferences that people make automatically and unconsciously and then utilize consistently, such as the selection of a polysemous word’s meaning, should not require elaborated, extended cognition. Even those who disagree with this desideratum should agree that if cognition in a transformer can be shifted from explicit thought traces to implicit activation dynamics, the resulting model will be more powerful.

3 Recurrent architectures

As we previously mentioned, the state tracking ability of a feedforward model is limited by model depth [e.g., 59, 61, 83] and by the fact that effectively utilizing the state representation becomes more challenging as it shifts upwards to deeper layers [9, 47, 79, 89].

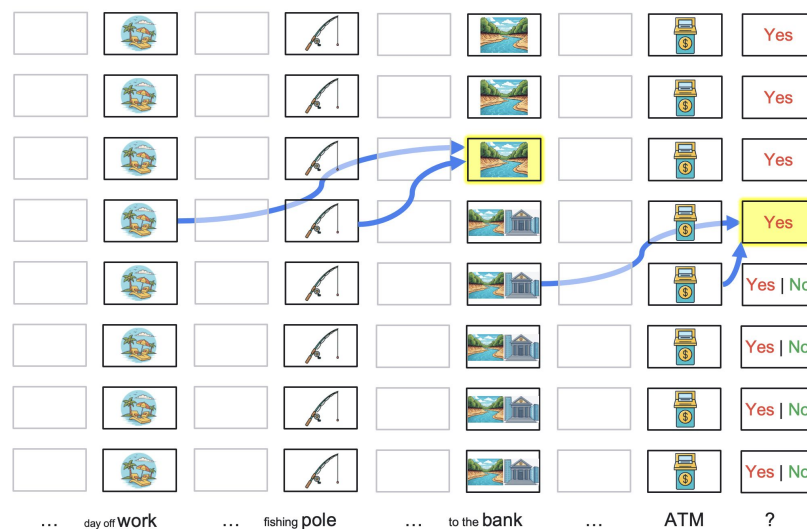


FIGURE 3 The depth of a state representation in a transformer can limit its utility for inference (adapted from Lepori et al., 2025). The input tokens are presented at the bottom, processed sequentially by the layers of the model. The symbols within the layers represent the model’s internal belief state. This highlights that despite the model converging to the correct belief (river bank) deep in the network at the “bank” token, this high-level disambiguation is inaccessible to the shallower layers of subsequent tokens. Consequently, when processing the final tokens, the model defaults to superficial associations (money bank → ATM) early in the stack, resulting in an incorrect prediction at the last token position.

The alternative is a recurrent model, which is necessary to express arbitrary state dynamics, i.e., $s_t = f(s_{t-1}, x_t)$. In this section, we explore how to combine recurrence with transformers. Given the subtle yet critical differences among varieties of recurrence, they are easily conflated and mistakenly treated as equivalent. This equivalence is problematic because recurrence is not in itself sufficient for state tracking; the most popular form of recurrence is unable to track state [62].

Figure 4, adapted from Rumelhart et al. [76], shows a simple recurrent net (left) and the net unrolled t steps to form a weight-constrained feedforward net (right). Unrolling the net requires making a copy of all neurons in the network for each step of recurrence. At each step, activation flows through each connection.

Now consider a transformer with some recurrent connections, as depicted in Figure 5a. As before, each box denotes a transformer block and the horizontal and vertical axes correspond to input tokens and layers, respectively. The additional arrows are meant to illustrate one possible type of recurrence involving activation flow from a deep layer to a shallow layer at every input step. The nature of the activation flow is not important for our purposes; for example, the shallow layer may generate queries that allow it to cross attend to keys and values from the deep layer [e.g., 20].

Unlike the simple RNN in Figure 4, which can be unrolled in only one way, unrolling the recurrent transformer in Figure 5a results in ambiguity due to the fact that the transformer architecture incorporates three distinct ordered dimensions: (1) the layers of the architecture,

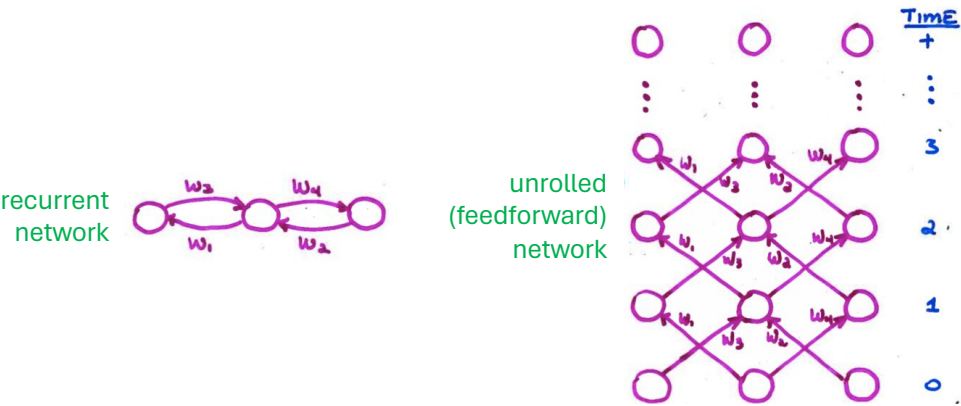


FIGURE 4 Unrolling a three-neuron recurrent neural net (left) into its equivalent feedforward network (right). Figure adapted from Rumelhart et al. [76].

bottom to top (shallow to deep) in the Figure; (2) the input steps of the architecture, left to right in the Figure; and (3) autoregressive steps performed at execution. During model pretraining, an ordinary (feedforward) transformer has only a single autoregressive step because all input

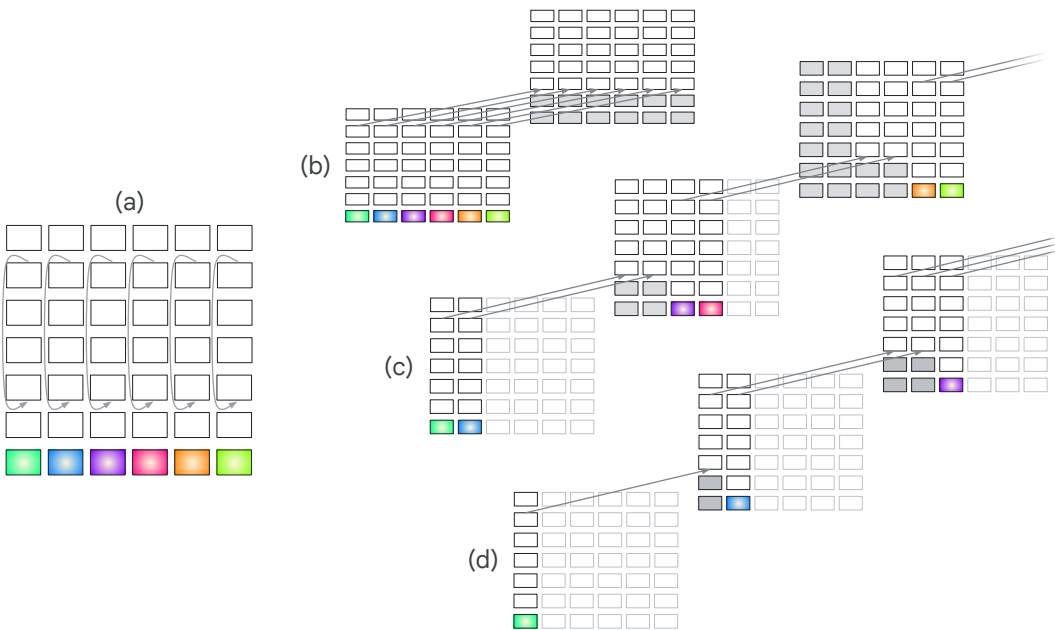


FIGURE 5 Unrolling a transformer. Each rectangle represents a transformer layer. The colored boxes represent distinct input tokens. The arrows indicate flow of activity. Note that the figure represents the output of the full transformer layer, which includes both the self-attention as well as the feedforward layer. The gray filled rectangles indicate blocks that can be replaced by the frozen KV cache as their values no longer change. The light grey unfilled rectangles represent input steps that are ignored at a given unrolling step. (a) A transformer with recurrence that allows activation from a deep layer to influence a shallow layer. (b) An unrolled depth-recurrent transformer. (c) An unrolled blockwise-recurrent transformer. (d) A transformer with potential attractor dynamics.

steps are run in parallel; and during inference, autoregressive steps are typically confounded with input steps, although multi-token prediction allows for multiple input/output steps per autoregressive step [25]. However, recurrence allows for further decoupling. In particular, many forms of recurrence require that even when a model is trained via teacher forcing, it must still be unrolled autoregressively [86]. This necessary sequentiality is what we mean by *autoregressive unrolling*, not the sequentiality that arises from token-by-token generation in a pure feedforward model.

Figure 5b depicts a transformer unrolled in depth, sometimes referred to as a *looped transformer*. Depth recurrence—whether of individual layers or ranges of layers, and whether deterministic or adaptive—is a very popular and successful approach. Some methods are designed and trained to allow for inference time scaling [e.g., 1, 5, 13, 22, 38, 67, 74, 75, 92, 97–99]; others incorporate recurrence via pretraining [77] or fine tuning a pretrained model [45, 57]; and surprisingly, several operate purely as an inference-time method to improve reasoning [12, 51, 66].

While depth recurrence can increase the expressivity of a transformer [78], it does not enable indefinite state tracking; the propagation is still depth limited. To appreciate this fact, pick any layer l in the lower or upper stack as the representation of $s(0)$, the state at the first stack. Then note that any $s(t+1)$, if it is to depend arbitrarily on $s(t)$ recursively, must be in a higher layer. The state representation still shifts upward due to the parallel propagation of activation across steps t , regardless of how deep the transformer is made to be with recurrent depth (Figure 1b).

Indefinite tracking of state with an arbitrary state update function requires sequential dependency that precludes parallelization across the sequence length *during training*. Two examples of autoregressive updates are presented in Figures 5c,d. Figure 5c depicts a blockwise-recurrent model [10, 14, 15, 36] in which a subsequence of input steps is run in parallel (two in the Figure) followed by an autoregressive iteration; Figure 5d shows a model in which one input step is presented per autoregressive step, and at step t , all stacks up to $t-1$ send a signal from the deep layer to the shallow layer, yielding a fully recurrent model. This model may have attractor dynamics since each layer continues to update, converging only when all previous steps have reached asymptote.

Having addressed recurrence with multiple input steps per autoregressive step (Figure 5c) and a standard one-to-one mapping (Figure 5d), we can also consider a setup where multiple

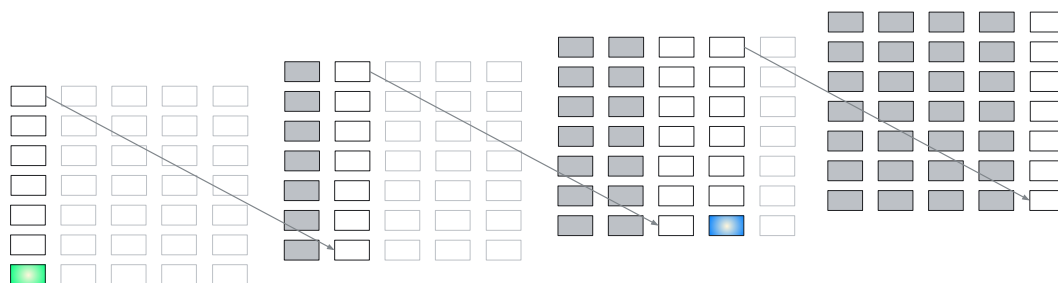


FIGURE 6 Unrolling a latent-thought model, where the model feeds back its latent thoughts as input to the model for multiple auto-regressive steps before processing the next input token, which is marked with the blue color [e.g., 29, 40].

Recurrence Axis	Input Tokens Per Recurrence Step		
	$Ratio > 1$	$Ratio = 1$	$Ratio < 1$
<i>Depth</i>	[Figure 5b] looped transformer [24], universal transformer [19], RINS [1]	[Figure 5d]	
<i>Step</i>	block-recurrent transformers [36]	[Figure 7b] linear attention [42], DeltaNet [80], MAMBA [28], canon layers [2], PaTH attention [95], RWKV-7 [70], test-time regression [84]	DeltaProduct [82]
<i>Depth + Step</i>	[Figure 5c] recurrent memory transformer [11], RINs [37], sentence gestalt [10]	feedback transformer [20]	[Figure 6] COCONUT [29], hierarchical reasoning model [40], CYB [21]

TABLE 1 A taxonomy of recurrent transformer architectures, with selected examples. Note that ratio > 1 indicates processing multiple tokens per recurrence step, while ratio < 1 indicates multiple recurrence steps per token.

autoregressive steps are executed for each input step (Figure 6). Latent-thought models have this form [e.g., 29, 40]. Some of these models can track state, whereas others do not [e.g., 21].

The examples of recurrence we have given all involve signal propagation from deeper layers to shallower layers, but within-layer propagation over inputs steps (Figure 7) also yields state dynamics [e.g., 2, 28].

Table 1 attempts to lay out a taxonomy of recurrent transformer architectures that includes the cases we have considered so far. We characterize architectures along two dimensions: *recurrence axis* and *input tokens per recurrence step*. The axis can be depth alone (Figures 5b,c,d), in step alone (Figure 7), and in both depth and step (Figure 6). The ratio of input tokens to recurrence steps can be greater than one (Figure 5b,c), equal to one (Figure 5d, 7), or less than one (Figure 6).³ In cells of the taxonomy, we list some popular and representative transformer-based architectures. We omit architectures that are described as “recurrent” in the sense that

³We define a ‘recurrence step’ strictly as a sequential dependency that precludes parallelization across the sequence length during training.

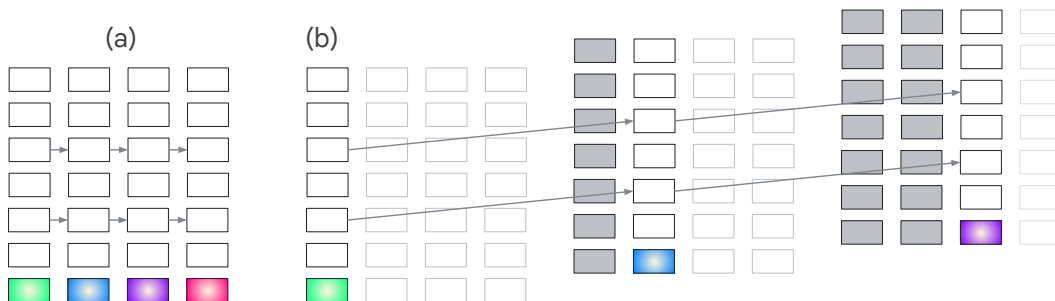


FIGURE 7 Unrolling an SSM, where information from the previous input step at layer l flows directly to the next step in the horizontal direction [e.g., 2, 28].

processing is iterative, but that are neither recurrent in depth nor step, e.g., Transformer-XL [17]. Being recurrent in depth and/or step is necessary for state tracking but is not sufficient. Essentially, full-fledged state tracking requires sequential dynamics during training; any model that can be entirely parallelized across the context has limitations in updating state.

While we have not shown all such architectures, we have not succeeded in identifying any examples of work that lies in the empty cells of the taxonomy. Some of these cells would be worth exploration. For example, architectures in the second row, third column, could include ones with within-layer attractor dynamics, where the model’s activation may iterate to convergence before advancing to the next token. Architectures in the first row, second and third columns, are interesting because they feed activation from a deep layer to a shallow layer (once for ratio = 1, multiple times for ratio > 1) in a manner that—unlike the looped transformer—does allow for indefinite state propagation.

We have deliberately sidestepped the question of what a recurrent arrow signifies in terms of activation dynamics. Different models make different proposals. The arrow could indicate copying key-value cache [96], it could represent a source of keys and values for cross attention [20] or self attention [69], or it might represent a direct connection through a linear layer, an MLP, or an adapter such as LoRA [33]. The coupling of step-to-step recurrence with attention—which gives neurons direct visibility to the entire step history—prevents credit assignment bottlenecks that arise when training traditional recurrent neural networks [43, 69].

4 Architectural limitations and workarounds

Liu et al. [56] point to the weakness of modern massively parallel architectures on problems that are inherently sequential, problems where combinatorics make it impractical to parallelize, such as state tracking, multihop inference, and planning. Formal analyses point to bounds on serial capacity for transformers [59]. State-space models (SSMs) are often touted as a means of state propagation, but SSMs with linear updates are no more expressive than an ordinary transformer [62]. In contrast, chain of thought does enhance model expressivity [50, 60], as one would intuit: Allowing a model to talk to itself, whether in natural language or latent space, sends signals from deep in the transformer to shallow layers, thereby propagating state forward. It is no wonder that frontier models have become increasingly reliant on internal ‘thinking’. However, the reliance on intermediate outputs to track micro-state may perform wasteful computation steps and unnecessarily consume the context window. Implicit activation dynamics—albeit recurrent—might be adequate to efficiently update mundane state information of the sort that people can process unconsciously and automatically. In human terms, it is fine to talk to yourself if you are reasoning through a calculus problem, but it is a bit weird to have to continually remind yourself of the relationship between two characters in a book you are reading.

Given their limitations in state tracking, why are transformers as successful as they are? The short answer is that by being able to reexamine their entire input history, they can often turn a state-tracking problem into a working memory problem, i.e., into retrieval from the context window. For example, consider the *latch* problem from the 1990s that required a sequence-processing model latch on to a bit of information early in a sequence and to retain it over a

long time gap [8, 64]. This problem in large part motivated LSTM but is entirely trivial for a transformer that can re-index into its input history to retrieve the early information. Transformers learn many clever strategies including this sort of lookback [73], associative scans [49], more specialized algorithms for formal language understanding [3, 72], including belief updating that reflects state uncertainty [81]. A simple illustration is the computation of parity from an input sequence of 0's and 1's. Instead of maintaining a parity state, the model can compute pairwise parity in the first layer of a transformer, then combine the pairs into four-bit parity in the second layer, and so forth, achieving a computation from scratch in $\log_2 n$ feedforward layers for a sequence of maximum length n . Such alternate and even shortcut solutions are common for transformers to develop in practice [55].

Another factor in the success of transformers is their support of *state compositionality*. State need not be—as we have depicted it in Figure 1—a monolithic representation. The representation of state can be split across embeddings and updated asynchronously for each component. For example, if the model needs to track the changing locations of two entities, those state variables can be updated independently for each entity.

5 Promising directions

In addition to the native mechanisms that transformers have to estimate state, several emerging research directions seem particularly promising for promoting state maintenance and updating. These directions balance the need for expressivity with computational feasibility.

5.1 Enhanced State-Space Models

While most linear SSMs do not exceed the expressivity of standard transformers, Delta Net [80], when its eigenvalue range is extended to include negative values [27], maintains the highly desirable property of being trainable in parallel while simultaneously achieving greater expressivity than a standard transformer. The delta rule underlying Delta Net has inspired a range of further developments, including RWKV-7 [70] and PaTH attention [95], that also achieve state tracking beyond the capability of ordinary transformers while demonstrating competitive language modeling at scale. Similarly, other new forms of attention are being developed with sequential dependencies that promise to be more powerful in stateful tasks [7, 48, 53], including the notion of gated linear attention [93] and gated Delta Net [94] that, when mixed with standard transformer blocks is more powerful than either, both in theory and practice [63].

5.2 Approximating state tracking in feedforward transformers

Rather than incorporating recurrence, feedforward transformers can be steered to better approximate state tracking through specialized training objectives and structural priors [34, 35, 85]. The promise is that such biases will encourage models to bolster their native lookback abilities. However, we hope that future research will take into account structured, compositional state representations.

5.3 Coarse recurrence

Introducing recurrence at a coarser granularity than individual tokens can mitigate the computational burden of token-by-token state updates. In the past, block-recurrent models have operated on this principle by compressing and passing memory forward in fixed-length chunks. A promising approach is to consider linguistic structure in identifying the chunks: Borazjanizadeh and McClelland [10] have modeled language as a sequence of discrete ‘thoughts’ by chunking at the sentence level.

5.4 Leveraging representational alignment

Variable-depth models dynamically select the number of iterations of a layer or the number of times a range of layers repeats. The fact that this approach succeeds with only fine tuning—and sometimes with no training whatsoever—suggests that the model is well predisposed to communication of representations across layers, due to the alignment resulting from the residual connections. Similarly, canon layers [3] leverage alignment of representations from one input step to the next. We speculate that there are additional means of leveraging this alignment.

5.5 Efficient training of recurrence

Any architecture which is capable of indefinite, arbitrary state propagation requires autoregressive processing. Autoregressive pretraining is computationally inefficient and limits parallelization [15]. One potential solution is a multi-stage training scheme in which initial pretraining relies entirely on standard, parallelizable feedforward transformer architectures. Recurrent mechanisms are then introduced only at a later training stage. To ensure training efficiency during these subsequent recurrent stages, optimization techniques such as truncated gradient methods and—for training attractor dynamics—recurrent backpropagation [4, 52, 71]. Methods have also been proposed to increase *arithmetic intensity* to obtain scaling that is near linear time in context length versus quadratic for a naive implementation [69].

6 Conclusions

Although the transformer’s feedforward design has expanded the limits of context-based retrieval, its topological structure remains fundamentally at odds with the iterative nature of state tracking. As we have argued, the current reliance on explicit natural-language-like “thought” to bypass depth limitations is an inefficient workaround for a structural deficiency. By transitioning toward implicit, recurrent activation dynamics, we can move beyond these depth-limited constraints to attain robust long-term coherence and multihop inference.

The taxonomy and research directions proposed in this article provide a roadmap for improving sequential inference dependencies without sacrificing the foundational strengths of modern models. Ultimately, bridging the gap between the transformer’s parallel efficiency and the brain’s inherent dynamical nature is essential. The next generation of foundation models must do more than simply re-scan the past; they must maintain a fluid, evolving representation of reality that

persists across the many time scales required for temporally extended cognition. Many thanks to Sunny Sanyal, Kazuki Irie, Kevin Murphy, Jay McClelland, and Chris Williams for helpful feedback on earlier drafts of the manuscript.

References

- [1] Ibrahim Alabdulmohsin and Xiaohua Zhai. Recursive inference scaling: A winning path to scalable inference in language and multimodal systems. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=cLbGkINOLP>.
- [2] Zeyuan Allen-Zhu. Physics of language models: Part 4.1, architecture design and the magic of canon layers, 2025. URL <https://arxiv.org/abs/2512.17351>. arXiv:2512.17351 [cs.CL].
- [3] Zeyuan Allen-Zhu and Yuanzhi Li. Physics of language models: Part 1, learning hierarchical language structures, 2025. URL <https://arxiv.org/abs/2305.13673>. arXiv:2305.13673 [cs.CL].
- [4] Luis B. Almeida. A learning rule for asynchronous perceptrons with feedback in a combinatorial environment. In *Proceedings of the IEEE First International Conference on Neural Networks*, volume 2, pages 609–618, San Diego, CA, USA, jun 1987. IEEE.
- [5] Sangmin Bae, Yujin Kim, Reza Bayat, Sungnyun Kim, Jiyouon Ha, Tal Schuster, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Aaron Courville, and Se-Young Yun. Mixture-of-recursions: Learning dynamic recursive depths for adaptive token-level computation. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=QuqsEIVWIG>.
- [6] Davide Baldelli, Ali Parviz, Amal Zouaq, and Sarath Chandar. Llms can’t play hangman: On the necessity of a private working memory for language agents, 2026. URL <https://arxiv.org/abs/2601.06973>. arXiv:2601.06973 [cs.CL].
- [7] Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- [8] Y. Bengio, P. Simard, and P. Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166, 1994. doi: 10.1109/72.279181.
- [9] Eden Biran, Daniela Gottesman, Sohee Yang, Mor Geva, and Amir Globerson. Hopping too late: Exploring the limitations of large language models on multi-hop queries. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 14113–14130, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.781. URL <https://aclanthology.org/2024.emnlp-main.781/>.

- [10] Nasim Borazjanizadeh and James McClelland. Modeling language as a sequence of thoughts, 2025. URL <https://arxiv.org/abs/2512.25026>. arXiv:2512.25026 [cs.CL].
- [11] Aydar Bulatov, Yury Kuratov, and Mikhail Burtsev. Recurrent memory transformer. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 11079–11091. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/47e288629a6996a17ce50b90a056a0e1-Paper-Conference.pdf.
- [12] Lizhang Chen, Jonathan Li, Chen Liang, Ni Lao, and Qiang Liu. Training-free looped transformers, 2026. URL <https://arxiv.org/abs/2605.23872>. arXiv:2605.23872 [cs.CL].
- [13] Yilong Chen, Junyuan Shang, Zhenyu Zhang, Yanxi Xie, Jiawei Sheng, Tingwen Liu, Shuo-huan Wang, Yu Sun, Hua Wu, and Haifeng Wang. Inner thinking transformer: Leveraging dynamic depth scaling to foster adaptive internal thinking, 2025. arXiv:2502.13842 [cs.CL].
- [14] Yinpeng Chen, DeLesley Hutchins, Aren Jansen, Andrey Zhmoginov, David Racz, and Jesper Sparre Andersen. MELODI: Exploring memory compression for long contexts. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=TvGPP8i18S>.
- [15] Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. Adapting language models to compress contexts, 2023. URL <https://arxiv.org/abs/2305.14788>. arXiv:2305.14788 [cs.CL].
- [16] Lonnie Chrisman. Reinforcement learning with perceptual aliasing: the perceptual distinctions approach. In *Proceedings of the Tenth National Conference on Artificial Intelligence*, AAAI’92, page 183–188. AAAI Press, 1992. ISBN 0262510634.
- [17] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-xl: Attentive language models beyond a fixed-length context, 2019. URL <https://arxiv.org/abs/1901.02860>. arXiv:1901.02860 [cs.LG].
- [18] Tim R. Davidson, Adam Fourney, Saleema Amershi, Robert West, Eric Horvitz, and Ece Kamar. The collaboration gap, 2025. URL <https://arxiv.org/abs/2511.02687>. arXiv:2511.02687 [cs.AI].
- [19] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. Universal transformers. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=HyzdRiR9Y7>.
- [20] Angela Fan, Thibaut Lavril, Edouard Grave, Armand Joulin, and Sainbayar Sukhbaatar. Addressing some limitations of transformers with feedback memory, 2021. URL <https://arxiv.org/abs/2002.09402>. arXiv:2002.09402 [cs.CL].
- [21] Alexandre Galashov, Matt Jones, Rosemary Ke, Yuan Cao, Vaishnavh Nagarajan, and Michael C. Mozer. Catch your breath: Adaptive computation for self-paced sequence production, 2025. URL <https://arxiv.org/abs/2510.13879>. arXiv:2510.13879 [cs.CL].

- [22] Jonas Geiping, Sean Michael McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=S3GhJooWIC>.
- [23] Asma Ghandeharioun, Avi Caciularu, Adam Pearce, Lucas Dixon, and Mor Geva. Patchscopes: a unifying framework for inspecting hidden representations of language models. In *Proceedings of the 41st International Conference on Machine Learning*, ICML’24. JMLR.org, 2024.
- [24] Angeliki Giannou, Shashank Rajput, Jy-yong Sohn, Kangwook Lee, Jason D Lee, and Dimitris Papailiopoulos. Looped transformers as programmable computers. In *International Conference on Machine Learning*, pages 11398–11442. PMLR, 2023.
- [25] Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. Better and faster large language models via multi-token prediction. *arXiv preprint arXiv:2404.19737*, 2024.
- [26] Satchel Grant, Noah D. Goodman, and James L. McClelland. Emergent symbol-like number variables in artificial neural networks, 2025. URL <https://arxiv.org/abs/2501.06141>.
- [27] Riccardo Grazi, Julien Siems, Arber Zela, Jörg K. H. Franke, Frank Hutter, and Massimiliano Pontil. Unlocking state-tracking in linear rnns through negative eigenvalues, 2025. URL <https://arxiv.org/abs/2411.12537>. arXiv:2411.12537 [cs.LG].
- [28] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=tEYskw1VY2>.
- [29] Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason E Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=Itxz7S4Ip3>.
- [30] Sepp Hochreiter. The vanishing gradient problem during learning recurrent neural nets and problem solutions. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 6(02):107–116, 1998.
- [31] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [32] Sepp Hochreiter, Yoshua Bengio, Paolo Frasconi, and Jürgen Schmidhuber. Gradient flow in recurrent nets: the difficulty of learning long-term dependencies, 2001.
- [33] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Liang Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2):3, 2022.

- [34] Edward S. Hu, Kwangjun Ahn, Qinghua Liu, Haoran Xu, Manan Tomar, Ada Langford, Dinesh Jayaraman, Alex Lamb, and John Langford. The belief state transformer. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=ThRMTcGpvo>.
- [35] Hai Huang, Yann LeCun, and Randall Balestriero. Semantic tube prediction: Beating LLM data efficiency with JEPA, 2026. URL <https://arxiv.org/abs/2602.22617>. arXiv:2602.22617 [cs.LG].
- [36] DeLesley Hutchins, Imanol Schlag, Yuhuai Wu, Ethan Dyer, and Behnam Neyshabur. Block-recurrent transformers. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 33248–33261. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/d6e0bbb9fc3f4c10950052ec2359355c-Paper-Conference.pdf.
- [37] Allan Jabri, David Fleet, and Ting Chen. Scalable adaptive computation for iterative generation, 2023. URL <https://arxiv.org/abs/2212.11972>. arXiv:2212.11972 [cs.LG].
- [38] Ahmadreza Jeddi, Marco Ciccone, and Babak Taati. Loopformer: Elastic-depth looped transformers for latent reasoning via shortcut modulation. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=RzYXb5YWBS>.
- [39] P.N. Johnson-Laird. *Mental Models: Towards a Cognitive Science of Language, Inference, and Consciousness*. Cognitive science series. Harvard University Press, 1983. ISBN 9780674568822. URL <https://books.google.com/books?id=FS3zSKAfLGMc>.
- [40] Alexia Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks, 2025. URL <https://arxiv.org/abs/2510.04871>. arXiv:2510.04871 [cs.LG].
- [41] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1):99–134, 1998. ISSN 0004-3702. doi: [https://doi.org/10.1016/S0004-3702\(98\)00023-X](https://doi.org/10.1016/S0004-3702(98)00023-X). URL <https://www.sciencedirect.com/science/article/pii/S000437029800023X>.
- [42] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 5156–5165. PMLR, 2020.
- [43] Nan Rosemary Ke, Anirudh Goyal ALIAS PARTH GOYAL, Olexa Bilaniuk, Jonathan Binas, Michael C Mozer, Chris Pal, and Yoshua Bengio. Sparse attentive backtracking: Temporal credit assignment through reminding. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc., 2018. URL https://proceedings.neurips.cc/paper_files/paper/2018/file/e16e74a63567ecb44ade5c87002bb1d9-Paper.pdf.

- [44] Arpandeeep Khatua, Hao Zhu, Peter Tran, Arya Prabhudesai, Frederic Sadrieh, Johann K. Lieberwirth, Xinkai Yu, Yicheng Fu, Michael J. Ryan, Jiaxin Pei, and Diyi Yang. Cooperbench: Why coding agents cannot be your teammates yet, 2026. URL <https://arxiv.org/abs/2601.13295>.
- [45] Yeskendir Koishckenov, Aldo Lipani, and Nicola Cancedda. Encode, think, decode: Scaling test-time reasoning with recursive latent thoughts, 2025. URL <https://arxiv.org/abs/2510.07358>. arXiv:2510.07358 [cs.LG].
- [46] Philippe Laban, Hiroaki Hayashi, Yingbo Zhou, and Jennifer Neville. Llms get lost in multi-turn conversation, 2025. URL <https://arxiv.org/abs/2505.06120>. arXiv:2505.06120 [cs.CL].
- [47] Michael A. Lepori, Michael Curtis Mozer, and Asma Ghandeharioun. Racing thoughts: Explaining contextualization errors in large language models. In Luis Chiruzzo, Alan Ritter, and Lu Wang, editors, *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 3020–3036, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. ISBN 979-8-89176-189-6. doi: 10.18653/v1/2025.naacl-long.155. URL <https://aclanthology.org/2025.naacl-long.155/>.
- [48] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Selective attention improves transformer. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=v0FzmPCd1e>.
- [49] Belinda Z. Li, Zifan Carl Guo, and Jacob Andreas. (how) do language models track state? In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=8SXosAVIFH>.
- [50] Zhiyuan Li, Hong Liu, Denny Zhou, and Tengyu Ma. Chain of thought empowers transformers to solve inherently serial problems. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=3EWTEy9MTM>.
- [51] Ziyue Li, Yang Li, and Tianyi Zhou. Skip a layer or loop it? test-time depth adaptation of pretrained llms, 2025. URL <https://arxiv.org/abs/2507.07996>. arXiv:2507.07996 [cs.LG].
- [52] Renjie Liao, Yuwen Xiong, Ethan Fetaya, Lisa Zhang, KiJung Yoon, Xaq Pitkow, Raquel Urtasun, and Richard Zemel. Reviving and improving recurrent back-propagation. In Jennifer Dy and Andreas Krause, editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 3082–3091. PMLR, 10–15 Jul 2018. URL <https://proceedings.mlr.press/v80/liao18c.html>.
- [53] Zhixuan Lin, Evgenii Nikishin, Xu He, and Aaron Courville. Forgetting transformer: Softmax attention with a forget gate. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=q2Lnyegkr8>.

- [54] Jack Lindsey, Wes Gurnee, Emmanuel Ameisen, Becca Chen, Adam Pearce, Nelson L. Turner, Chris Citro, et al. On the biology of a large language model. *Transformer Circuits Thread*, 2025. URL <https://transformer-circuits.pub/2025/attribution-graphs/biology.html>.
- [55] Bingbin Liu, Jordan T Ash, Surbhi Goel, Akshay Krishnamurthy, and Cyril Zhang. Transformers learn shortcuts to automata. *arXiv preprint arXiv:2210.10749*, 2022.
- [56] Yuxi Liu, Konpat Preechakul, Kananart Kuwarananchaoen, and Yutong Bai. The serial scaling hypothesis, 2026. URL <https://arxiv.org/abs/2507.12549>.
- [57] Sean McLeish, Ang Li, John Kirchenbauer, Dayal Singh Kalra, Brian R. Bartoldson, Bhavya Kailkhura, Avi Schwarzschild, Jonas Geiping, Tom Goldstein, and Micah Goldblum. Teaching pretrained language models to think deeper with retrofitted recurrence, 2025. URL <https://arxiv.org/abs/2511.07384>.
- [58] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in gpt. *Advances in neural information processing systems*, 35:17359–17372, 2022.
- [59] William Merrill and Ashish Sabharwal. The parallelism tradeoff: Limitations of log-precision transformers. *Transactions of the Association for Computational Linguistics*, 11:531–545, 2023. doi: 10.1162/tacl_a_00562. URL <https://aclanthology.org/2023.tacl-1.31/>.
- [60] William Merrill and Ashish Sabharwal. The expressive power of transformers with chain of thought. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=NjNGlPh8Wh>.
- [61] William Merrill and Ashish Sabharwal. A little depth goes a long way: The expressive power of log-depth transformers, 2025. URL <https://arxiv.org/abs/2503.03961>. arXiv:2503.03961 [cs.LG].
- [62] William Merrill, Jackson Petty, and Ashish Sabharwal. The illusion of state in state-space models, 2025. URL <https://arxiv.org/abs/2404.08819>. arXiv:2404.08819 [cs.LG].
- [63] William Merrill, Yanhong Li, Tyler Romero, Anej Svete, Caia Costello, Pradeep Dasigi, Dirk Groeneveld, David Heineman, Bailey Kuehl, Nathan Lambert, Chuan Li, Kyle Lo, Saumya Malik, DJ Matusz, Benjamin Minixhofer, Jacob Morrison, Luca Soldaini, Finbarr Timbers, Pete Walsh, Noah A. Smith, Hannaneh Hajishirzi, and Ashish Sabharwal. Olmo hybrid: From theory to practice and back. Technical report, Allen Institute for AI, 2026. URL <https://allenai.org/papers/olmo-hybrid>. Technical Report.
- [64] Michael C Mozer. Induction of multiscale temporal structure. *Advances in neural information processing systems*, 4, 1991.
- [65] Michael C. Mozer. The induction of multiscale temporal structure. In John E. Moody, Stephen J. Hanson, and Richard P. Lippmann, editors, *Advances in Neural Information Processing Systems 4*, pages 275–282, San Mateo, CA, 1992. Morgan Kaufmann.

- [66] David Noel Ng. LLM neuroanatomy: How I topped the LLM leaderboard without changing a single weight. <https://dnhkng.github.io/posts/rys/>, March 2026.
- [67] Aleksandra I. Nowak, Otniel-Bogdan Mercea, Anurag Arnab, Jonas Pfeiffer, Yann Dauphin, and Utku Evci. Towards optimal adapter placement for efficient transfer learning, 2024. URL <https://arxiv.org/abs/2410.15858>. arXiv:2410.15858 [cs.LG].
- [68] Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Scott Johnston, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. In-context learning and induction heads. *Transformer Circuits Thread*, 2022. <https://transformer-circuits.pub/2022/in-context-learning-and-induction-heads/index.html>.
- [69] Costin-Andrei Oncescu, Depen Morwani, Samy Jelassi, Alexandru Meterez, Mujin Kwun, and Sham Kakade. The recurrent transformer: Greater effective depth and efficient decoding, 2026. URL <https://arxiv.org/abs/2604.21215>. arXiv:2604.21215 [cs.LG].
- [70] Bo Peng, Ruichong Zhang, Daniel Goldstein, Eric Alcaide, Xingjian Du, Haowen Hou, Jiaju Lin, Jiaying Liu, Janna Lu, William Merrill, Guangyu Song, Kaifeng Tan, Saiteja Utpala, Nathan Wilce, Johan S. Wind, Tianyi Wu, Daniel Wuttke, and Christian Zhou-Zheng. RWKV-7 "goose" with expressive dynamic state evolution. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=ayB1PACN5j>.
- [71] Fernando Pineda. Generalization of back-propagation to recurrent neural networks. *Physical Review Letters*, 59(19):2229–2232, nov 1987. doi: 10.1103/PhysRevLett.59.2229.
- [72] Mateusz Piotrowski, Paul M. Riechers, Daniel Filan, and Adam S. Shai. Constrained belief updates explain geometric structures in transformer representations, 2025. URL <https://arxiv.org/abs/2502.01954>. arXiv:2502.01954 [cs.LG].
- [73] Nikhil Prakash, Natalie Shapira, Arnab Sen Sharma, Christoph Riedl, Yonatan Belinkov, Tamar Rott Shaham, David Bau, and Atticus Geiger. Language models use lookbacks to track beliefs. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=6gO6KTRMpG>.
- [74] David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models, 2024. URL <https://arxiv.org/abs/2404.02258>. arXiv:2404.02258 [cs.LG].
- [75] Ivan Rodkin, Daniil Orel, Konstantin Smirnov, Arman Bolatov, Bilal Elbouardi, Beshar Hassan, Yuri Kuratov, Aydar Bulatov, Preslav Nakov, Timothy Baldwin, Artem Shelmanov, and Mikhail Burtsev. Beyond memorization: Extending reasoning depth with recurrence, memory and test-time compute scaling, 2025. URL <https://arxiv.org/abs/2508.16745>. arXiv:2508.16745 [cs.LG].

- [76] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *nature*, 323(6088):533–536, 1986.
- [77] Sunny Sanyal. Looped-gpt: Looping during pre-training improves generalization, 2026. URL <https://sanyalsunny111.github.io/posts/2026-01-15-post1-looped-gpt/>.
- [78] Nikunj Saunshi, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J. Reddi. Reasoning with latent thoughts: On the power of looped transformers. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=din0lGfZFd>.
- [79] Daniel P. Sawyer, Nan Rosemary Ke, Hubert Soyer, Martin Engelcke, John Reid, David P Reichert, Drew A. Hudson, Alexander Lerchner, Danilo Jimenez Rezende, Timothy P Lillicrap, Michael Curtis Mozer, and Jane X Wang. Exploring exploration with foundation agents in interactive environments. In *NeurIPS 2025 Workshop on Embodied World Models for Decision Making*, 2025. URL <https://openreview.net/forum?id=ay2d66HW02>.
- [80] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. *arXiv preprint arXiv:2102.11174*, 2021.
- [81] Adam Shai, Paul M. Riechers, Lucas Teixeira, Alexander Gietelink Oldenziel, and Sarah Marzen. Transformers represent belief state geometry in their residual stream. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=YIB7REL8UC>.
- [82] Julien Siems, Timur Carstensen, Arber Zela, Frank Hutter, Massimiliano Pontil, and Riccardo Grazi. Deltaproduct: Improving state-tracking in linear RNNs via householder products. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=SoRiaijTGr>.
- [83] Lena Strobl, William Merrill, Gail Weiss, David Chiang, and Dana Angluin. What formal languages can transformers express? a survey. *Transactions of the Association for Computational Linguistics*, 12:543–561, 05 2024. ISSN 2307-387X. doi: 10.1162/tac1_a_00663. URL https://doi.org/10.1162/tac1_a_00663.
- [84] Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to (learn at test time): RNNs with expressive hidden states. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=wXfu0j9C7L>.
- [85] Jayden Teoh, Manan Tomar, Kwangjun Ahn, Edward S. Hu, Pratyusha Sharma, Riashat Islam, Alex Lamb, and John Langford. Next-latent prediction transformers learn compact world models, 2025. URL <https://arxiv.org/abs/2511.05963>.

- [86] Jayden Teoh, Manan Tomar, Kwangjun Ahn, Edward S Hu, Pratyusha Sharma, Riashat Islam, Alex Lamb, and John Langford. Next-latent prediction transformers learn compact world models. *arXiv preprint arXiv:2511.05963*, 2025.
- [87] Amos Tversky and Daniel Kahneman. Belief in the law of small numbers. *Psychological Bulletin*, 76(2):105–110, 1971. doi: 10.1037/h0031322.
- [88] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008. Curran Associates, Inc., 2017. URL <https://papers.nips.cc/paper/7181-attention-is-all-you-need>.
- [89] Constantin Venhoff, Ashkan Khakzar, Sonia Joseph, Philip Torr, and Neel Nanda. Too late to recall: Explaining the two-hop problem in multimodal knowledge retrieval. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=qeL8fi8GS7>.
- [90] Edward Vul, Noah Goodman, Thomas L. Griffiths, and Joshua B. Tenenbaum. One and done? Optimal decisions from very few samples. *Cognitive Science*, 38(4):599–637, 2014. doi: 10.1111/cogs.12101.
- [91] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- [92] Liu Yang, Kangwook Lee, Robert D Nowak, and Dimitris Papailiopoulos. Looped transformers are better at learning learning algorithms. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=HHbRxoDTxE>.
- [93] Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. In *Proceedings of the 41st International Conference on Machine Learning*, ICML’24. JMLR.org, 2024.
- [94] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=r8H7xhYPwz>.
- [95] Songlin Yang, Yikang Shen, Kaiyue Wen, Shawn Tan, Mayank Mishra, Liliang Ren, Rameswar Panda, and Yoon Kim. PaTH attention: Position encoding via accumulating householder transformations. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=ZBlHEeSvKd>.
- [96] Yifei Yang, Zouying Cao, Qiguang Chen, Libo Qin, Dongjie Yang, Hai Zhao, and Zhi Chen. Kvsharer: Efficient inference via layer-wise dissimilar kv cache sharing. *arXiv preprint arXiv:2410.18517*, 2024.

- [97] Zeping Yu, Yonatan Belinkov, and Sophia Ananiadou. Back attention: Understanding and enhancing multi-hop reasoning in large language models. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 11257–11272, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.567. URL <https://aclanthology.org/2025.emnlp-main.567/>.
- [98] Boyi Zeng, Shixiang Song, Siyuan Huang, Yixuan Wang, He Li, Ziwei He, Xinbing Wang, Zhiyu li, and Zhouhan Lin. PonderLM: Pretraining language models to ponder in continuous space. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=UrM4MNRYZm>.
- [99] Rui-Jie Zhu, Zixuan Wang, Kai Hua, Tianyu Zhang, Ziniu Li, Haoran Que, Boyi Wei, Zixin Wen, Fan Yin, He Xing, Lu Li, Jiajun Shi, Kaijing Ma, Shanda Li, Taylor Kergan, Andrew Smith, Xingwei Qu, Mude Hui, Bohong Wu, Qiyang Min, Hongzhi Huang, Xun Zhou, Wei Ye, Jiaheng Liu, Jian Yang, Yunfeng Shi, Chenghua Lin, Enduo Zhao, Tianle Cai, Ge Zhang, Wenhao Huang, Yoshua Bengio, and Jason Eshraghian. Scaling latent reasoning via looped language models, 2025. URL <https://arxiv.org/abs/2510.25741>. arXiv:2510.25741 [cs.LG].